

CAMP

Homework 3:

Report of Pipeline

< -2 free-days left >

Name : SeokBeom Lim (임석범)

Department : Mobile System Engineering

Student number : 32203743

Table of contents

1. 서론	2
2. 본론	
1) SingleCycle 과제 보완점	3
2) Single Cycle 및 Pipeline 개념	4
3) Pipeline 코드 설계 요소	4
4) 코드 결과 및 결과에 대한 수학적 고찰	5
3. 결론	24

서론

현대의 컴퓨터 구조에서 대부분의 컴퓨터는 성능 향상을 위해 파이프라인 기술을 사용 한다. 파이프라인 기술은 명령어 병렬처리 기술로 여러 명령어를 동시에 처리한다. 이번 과제를 통해 파이프라인 구조를 갖춘 MIPS 시뮬레이터를 구현하고 지난 과제의 Single cycle processor와 비교 분석하여 컴퓨터구조와 파이프라인 기술에 대한 이해를 높이고자 한다. 또한, 이번 보고서를 통해 저번 코드와 보고서의 완성도를 높이고 파이프라인 구조의 헤저드 처리를 위한 창의적 방법을 소개하고자 한다.

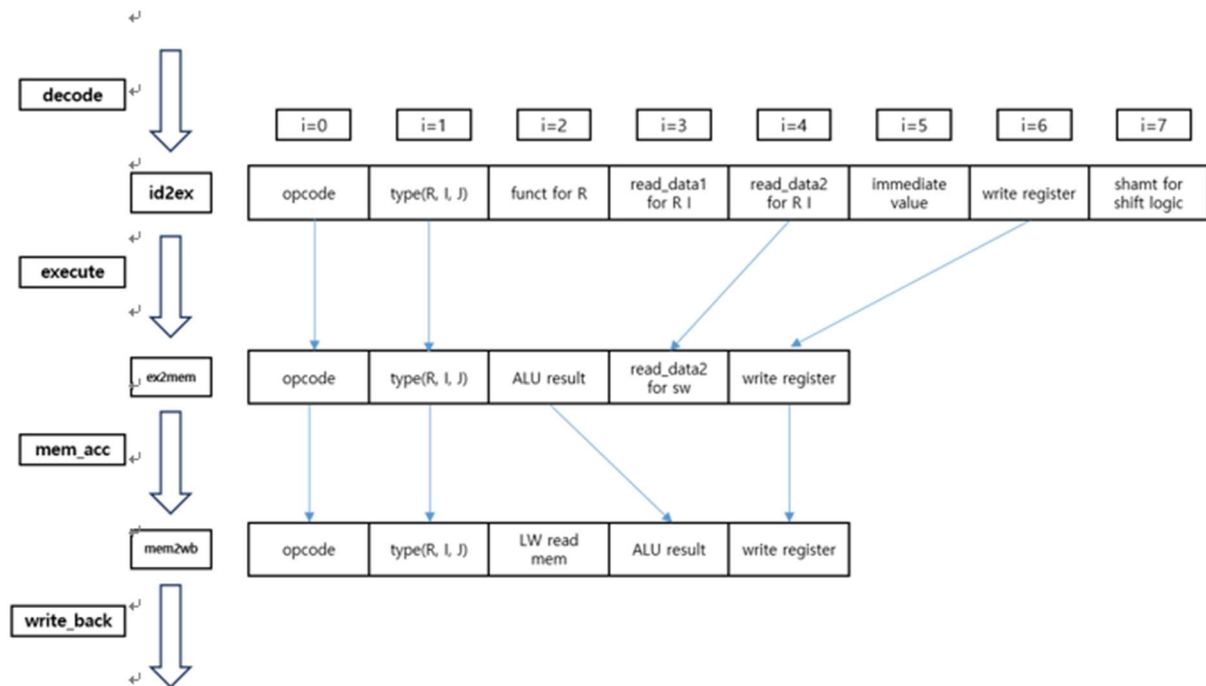
본론

I. Single Cycle 과제 보완점

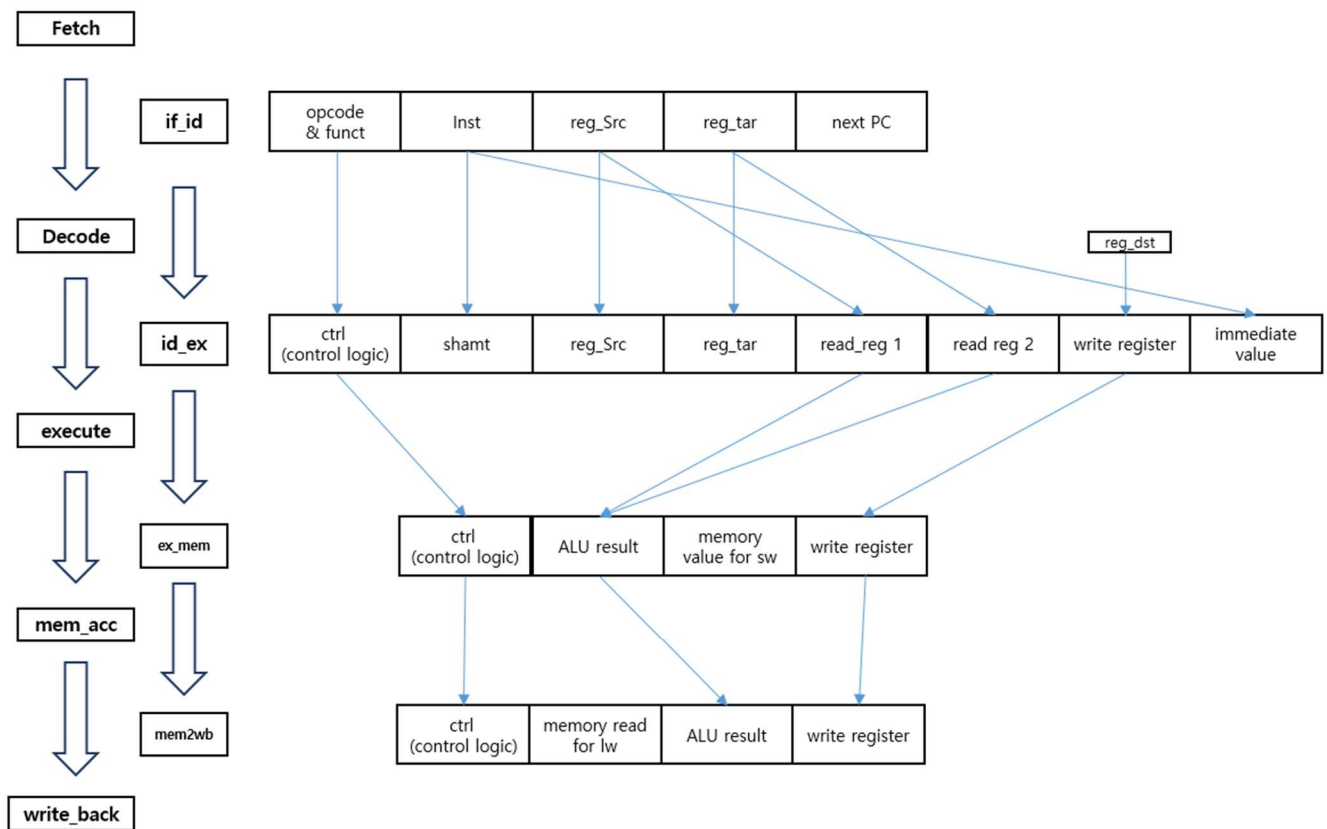
1) Latch의 가독성 보완

지난 과제에서 Latch를 이용하여 Single Cycle MIPS emulator 를 구현하였다. Single Cycles에서 Latch는 배열을 이용하였으며 배열의 개수는 5개에서 8개를 사용 하였다. 이러한 Latch의 사용은 전체적인 코드의 가독성을 떨어뜨리는 원인이 되었다. 이를 보완하고자, 구조체를 사용하였으며 구조체의 이동은 아래 그림을 통해 확인 할 수 있다. 또한, 아래 그림을 통해 지난 Single Cycle 과제의 Latch와 비교하면 Latch에 들어가는 값의 개수 또한 많아졌다.

[Single Cycle Latch]



[Pipeline Latch]



2) Control logic의 세분화

코드로 컴퓨터 구조를 구현 할때, Control Logic을 많이 쓸수록 함수에서 사용해야되는 조건문이 복잡해진다. 지난 과제를 구현 할 때 Control Logic을 5개 사용 하였으며, 이로 인해 전체적인 코드의 복잡도가 상승하였다. 이번 과제에서는 Control Logic의 수를 늘리고 세분화 하여 사용하여 코드의 복잡도를 낮추고자 한다. 다음은 Control Logic의 설명이다.

1. `alu_ctrl` : `alu_control`은 4비트로 이루어져 있으며, 값에 따라 ALU 에서 행해지는 계산이 달라진다. 예를 들어, `alu_ctrl` 값이 `0b0010`이면 `alu`는 `add`를 수행한다.
2. `mem_read` : Data memory를 읽기 위한 control 값이다. 1일 때, 메모리를 읽을 수 있다.
3. `mem_write` : Data memory 를 쓰기 위한 control 값이다. 1일 때 메모리를 쓸 수 있다.
4. `mem_to_reg` : LW의 경우, memory를 읽은 후, WB단계로 memory에서 읽은 값을 선택하는 control logic 이다.
5. `reg_wb` : Register에 값을 쓸 수 있게 하는 Control Logic이다. Hazard Detect Logic을 위해 필수적인 Control Logic이다.
6. `reg_dst` : ID_EX Latch에서 EX_MEM Latch로 넘어갈 때, `reg_write`의 값을 Rd를 받을 것인지 Rt를 받을 것인지 결정한다. Rtype과 Itype을 구별하는 Control Logic으로 사용하기도 했다.
7. `get_imm` : 이번 코드에서 새로 만든 변수이고, `get_imm` 값에 따라 immediate 값을 sign extend, zero extend, upper extend 할지 정한다.
8. `ex_skip` : Branch와 J 관련 명령어들을 Decode를 옮기게 되어 만들어진 Control Logic이다. Decode 이후에 필요가 없어진 위 명령어들을 skip 할 수 있도록 도와준다.

9. rs_ch : Data Hazard를 위해 만들어진 Control Logic 이며, 해당 명령어의 reg_src 값이 이전 명령어에 의한 변화된 값이 현재 명령어에 영향을 줄 수 있음을 의미한다. 예를 들어, SW 명령어의 경우 rs와 rt 값이 모두 변하면 안된다. 위의 경우, rs_ch 와 rt_ch 는 1이 된다.

10. rt_ch : rs_ch와 유사하며 해당 명령어의 reg_src 값이 이전 명령어에 의한 변화된 값이 현재 명령어에 영향을 줄 수 있음을 의미한다.

아래는 Control Logic을 명령어로 정리한 표이다.

	opcode	funct	reg_wb	get_imm	reg_dst	alu_ctrl	mem_read	mem_write	mem_to_reg	ex_skip	rt_ch	rs_ch
j	0x2	x	0	0	x	x	0	0	0	1	0	0
jal	0x3	x	0	0	x	x	0	0	0	1	0	0
beq	0x4	x	0	0	x	x	0	0	0	1	1	1
bne	0x5	x	0	0	x	x	0	0	0	1	1	1
addi	0x8	x	1	1	0	0b0010	0	0	0	0	0	1
addiu	0x9	x	1	1	0	0b0010	0	0	0	0	0	1
slti	0xa	x	0	1	0	0b0111	0	0	0	0	0	1
sltiu	0xb	x	0	1	0	0b0111	0	0	0	0	0	1
andi	0xc	x	0	2	0	0b0000	0	0	0	0	0	1
ori	0xd	x	0	2	0	0b0001	0	0	0	0	0	1
lw	0x23	x	1	1	0	0b0010	1	0	1	0	0	1
sw	0x2b	x	0	1	x	0b0010	0	1	0	0	1	1
lui	0xf	x	1	3	0	0	0	0	0	2	0	0
Add	0x0	0x20	1	0	1	0b0010	0	0	0	0	1	1
Subtract	0x0	0x22	1	0	1	0b0110	0	0	0	0	1	1
And	0x0	0x24	1	0	1	0b0000	0	0	0	0	1	1
Or	0x0	0x25	1	0	1	0b0001	0	0	0	0	1	1
Nor	0x0	0x27	1	0	1	0b1100	0	0	0	0	1	1
sll	0x0	0x00	1	0	1	0b1110	0	0	0	0	1	0
SRL	0x0	0x02	1	0	1	0b1111	0	0	0	0	1	0
slt	0x0	0x2a	1	0	1	0b0110	0	0	0	0	1	1
Sltu	0x0	0x2b	1	0	1	0b0111	0	0	0	0	1	1
jr	0x0	0x08	1	0	x	x	0	0	0	1	0	1
Subu	0x0	0x23	1	0	1	0b0110	0	0	0	0	1	1
addu	0x0	0x21	1	0	1	0b0010	0	0	0	0	1	1

3) Branch, Jump, Jal, Jr 의 보완

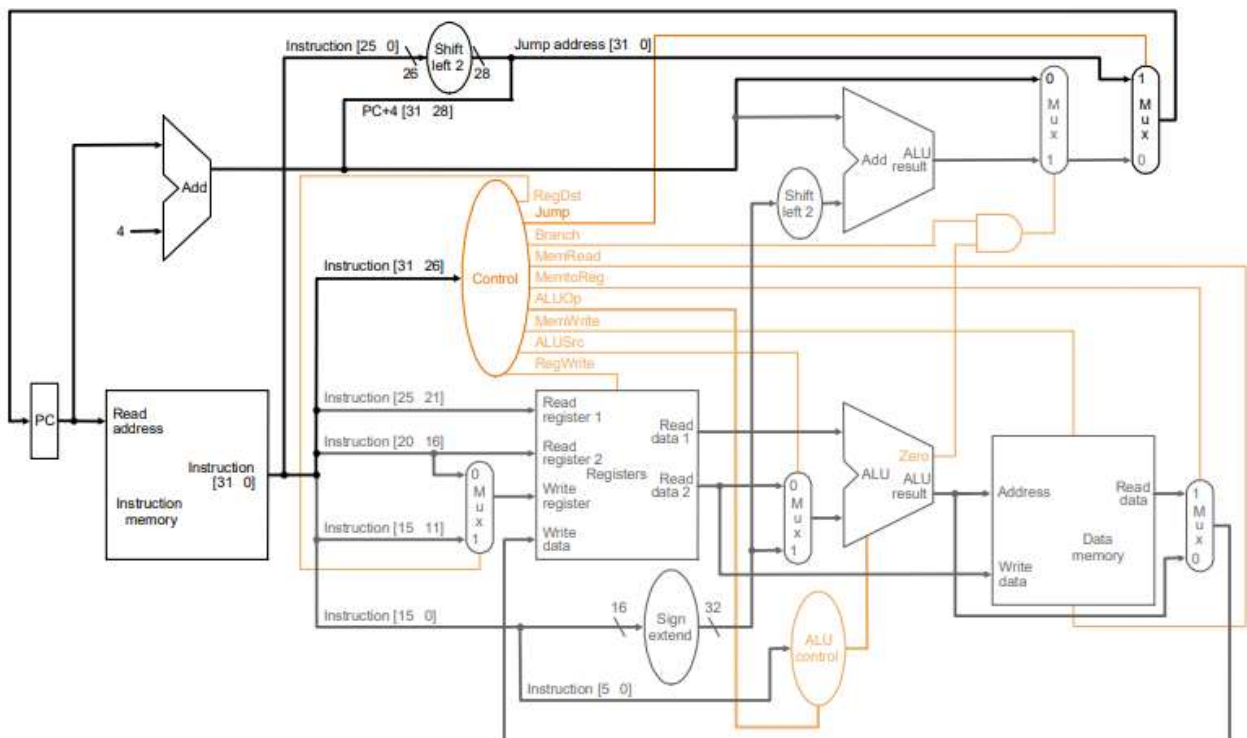
Single Cycle 코드에서는 Control Logic을 많이 사용하지 않아 복잡했었다. 이에 따라, Branch, Jump, Jal, Jr은 Execute단계가 끝나면 다음 명령어로 넘어가게 설정하였었다. Pipeline 코드는 Decode 단계로 이동된 명령어를 제외한 나머지 모든 명령어들은 모든 단계를 지나며 Decode로 이동된 명령어들은 ex_skip을 통해 execute와 memory access 단계를 Skip 한다.

(예시 : jr가 memory accesss 단계를 건너 뛴다.)

II. Single Cycle 및 Pipeline (Multi Cycle의 개념)

1) Single Cycle 개념

Single cycle Process는 한 사이클에서 명령어의 Fetch부터 Write Back 단계까지 모든 단계를 하나의 사이클에서 완료한다. 각 단계를 구체화 한 그림은 다음 그림과 같다.



2) Single Cycle Process 단계

모든 명령어는 하나의 사이클에서 마무리 되어야 하며 한 사이클에는 명령어 인출(IF), 명령어 해석(ID), 실행(Ex), 메모리 접근(MEM), 라이트 백(WB) 단계가 있다. 단계에 대한 설명은 다음과 같다.

1. 명령어 인출 (Instruction fetch, IF)
2. 명령어 해석 (Instruction Decode, ID)
3. 실행 (Execute, EX)
4. 메모리 접근 (Memory Access, MEM)
5. 라이트 백 (Write Back, WB)

명령어 인출에서 PC값을 통해 명령어를 읽어드리고(IF), 인출된 명령어는 ID단계로 보내진다. 보내진 명령어는 해독 가능한 값으로 쪼개지고 쪼개진 값에 따라 Control logic이 정해진다. 이후, EX단계에서 Decode된 명령어를 실행하고 Memory access 단계에서 LW와 SW를 통해 데이터를 적재 및 저장한다. 이후 WB 단계를 통해 Register 값을 갱신하면 하나의 사이클이 마무리 된다.

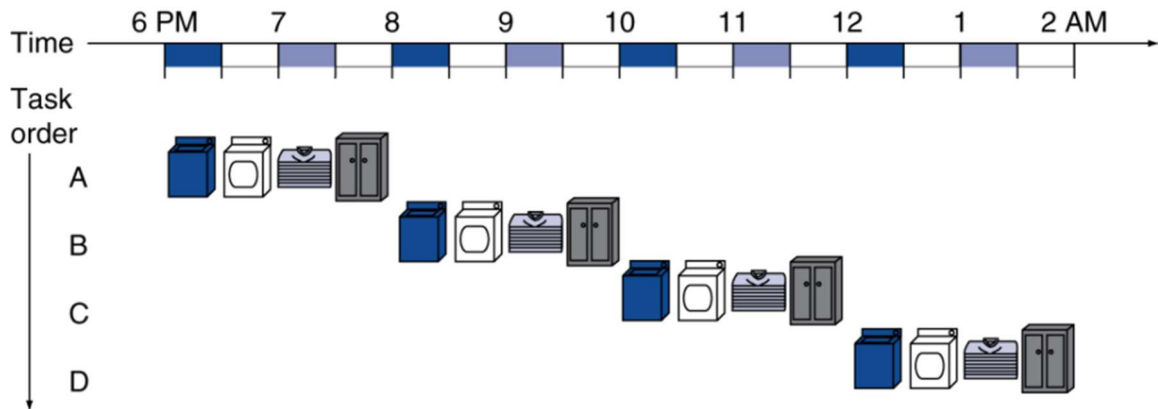
3) Single Cycle의 한계점

위와 같이 단일 사이클에서 하나의 명령어만 사용되면, 실행 시간이 가장 긴 명령어를 기준으로 연산이 처리되며 각 명령어를 처리할 때, 하나의 단계만 사용되어 다른 Micro Architecture를 사용 할 수 없어 효율성이 떨어진다. 이와 다르게, Multi Cycle 구조는 각 명령어를 병렬적으로 처리하여 Single Cycle Process 보다 뛰어난 효율을 가진다.

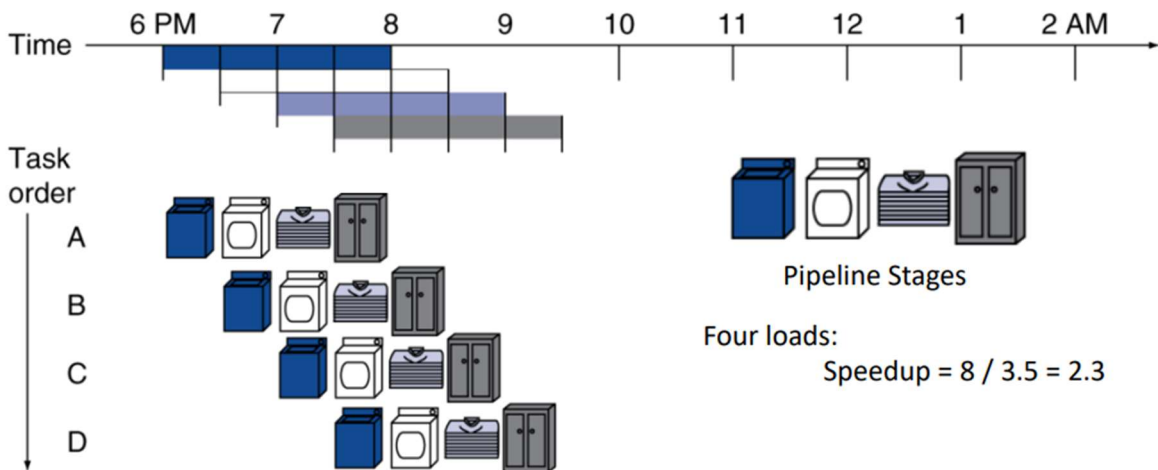
4) Multi Cycle 개념

Multi Cycle 구조는 Single Cycle과 같은 단계를 가지지만 하나의 명령어를 하나의 클럭 사이클에 사용하는 것이 아닌 여러 명령어를 하나의 클럭사이클에 사용 할 수 있게 한다.

아래 그림을 통해 파이프라인에 대해 이해 할 수 있다.



[Single Cycle]



[Pipeline]

빨래를 돌린다고 가정할 때, 세탁, 건조, 접기, 보관을 한다고 가정하자. 위 같은 순서는 바뀌지 않고, 같은 시간에 하나씩 처리하는 것보다, 4가지 단계를 같은 시간에 병렬적으로 처리하면 2.3 배 효율이 높아지는 것을 알 수 있다. 이러한 병렬 처리에 대한 개념은 Multi Cycle Process 에

서도 똑같이 적용된다.

5) Hazard

여러 명령이 동시에 실행 되면서 발생하는 문제인 Hazard는 명령어의 실행이 최신 값을 사용하지 못해 잘못된 결과를 초래하는 현상이다. 3가지 Hazard가 존재하며, Hazard는 다음과 같다.

1. Data Hazard

2. Control Hazard

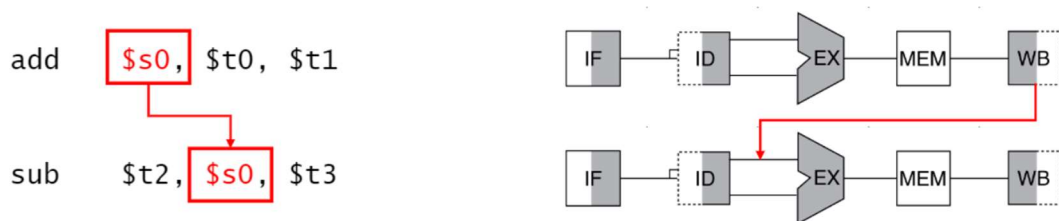
3. Structural Hazard

첫번째로 Data Hazard는 이전 명령어에서 변경된 값이 이후의 명령어에서 최신화 되지 못하여 발생한다. 이전 명령어에서 레지스터가 값을 쓰고 이후 명령어가 값을 읽을 때(RAW), 이후 명령어가 레지스터에 값을 수정하기 전에 명령어가 값을 읽을 때(WAR), 명령어들이 같은 레지스터 값을 쓰려고 할때(WAW) 발생한다. Data Hazard는 Forwarding과 Stalling을 통해 해결 할 수 있다.

두번째는 Control Hazard로 Branch 명령어가 나왔을 때, Branch의 taken 여부 결정 문제이다. Branch의 분기 여부에 따라 이후의 명령어의 주소를 결정하게 되는데, 보통 분기 예측(Branch Prediction) 을 통해 결정된다. Branch 분기 예측에 실패 하였을 때, 진행된 명령어들을 Flush한 후 다음 명령어를 진행한다.

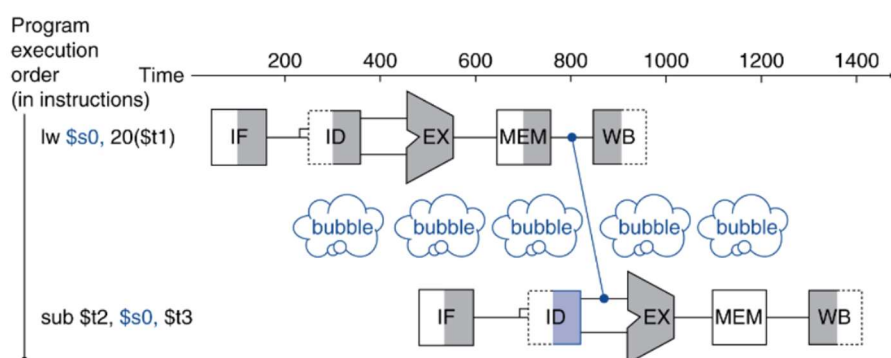
세번째는 Structural Hazard 이다. Structural Hazard는 하나의 하드웨어 자원에 대해 여러 명령어가 동시에 요구 할 때 발생한다. 구조적 Hazard의 경우, 레지스터 스케줄링을 통해서 명령어를 재배치 한후, 실행하면 해결된다.

위 세가지 Hazard중 Data Hazard, 와 Control Hazard 에 대해서만 이번 코드에서 다룬다. 두 Hazard의 해결방법은 Forwarding, Stalling, Branch Prediction이 있다. Forwarding은 사용할 레지스터 값이 변경되었다면 변경된 값을 보내주어 해결하는 방법이다.



위 사진을 보면 \$S0 레지스터가 첫번째 명령어에서 써지고(write) 다음 레지스터에서 읽어지는 것을 알 수 있다. 두번째 명령어에서는 첫번째 명령어의 레지스터 변화 값이 써지지 않았으므로, Alu Result 값을 두번째 명령어에 전달 해주는 방법으로 Hazard를 처리 할 수 있다.

Stalling은 주로 Load-use 에 사용 되며, 이전 명령어의 레지스터 및 메모리 값 변화를 기다려서 다음 명령어에서 최신 변화 값을 사용 하도록 하는 기법이다. 최신 변화값을 기다리는 단계를 주로 Bubble 또는 nop 이라 부르며 Bubble이 있는 동안 모든 Control logic을 0으로 값을 읽지도 쓰지도 않는 상태를 만든다. Instruction 00000000의 경우 sll r0 r0 0인데 이 경우 r0의 값은 변하지 않는 상수 이며 이때 모든 control logic을 0으로 만드는 것으로 Stall을 구현 할 수 있다.

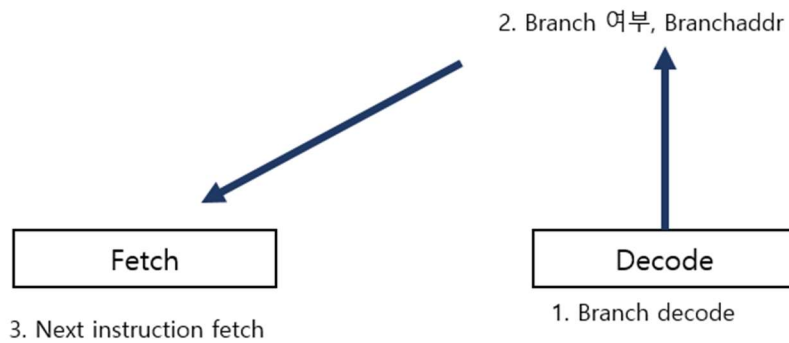


마지막으로 Branch prediction은 Computer architecture에서 8 great idea 중에 하나인 Hazard 처리 기법이다. 이에 대한 기본적인 생각은 미래에 발생할 예측을 준비하여 설계의 성능을 높이는 것이다. 예측 방법은 두가지로 나뉘는데, 간단한 Static Prediction 과 좀더 복잡한 Dynamic Prediction 이 있다. Static Prediction 의 경우 분기 방향이 고정되어 하나의 방향으로 예측한다. 이와 다르게 Dyanmaic prediction은 과거 실행 결과를 통해 미래를 예측하는 방식을 가진다. Dynamic Prediction의 예시로 1,2-bit predictor 가 있다. 물론 예측 기술은 8 great idea in computer arch 에 들 정도로 매력적인 기법이지만 잘못된 예측은 성능 저하를 유발하므로 이에 대한 정확도를 높이기 위한 여러 방법을 사용 해야 한다.

III. Pipeline 코드 설계 요소

1) 목적 및 전체적 개요.

이번 파이프라인 코드 설계는 이전 Single Cycle의 코드를 보완 시키고 성능이 높은 Pipeline을 구현하는 것에 최종 목표를 두었다. Pipeline의 성능을 높이기 위해서는 두가지가 중요하다. 첫째는, 최소한의 stall을 하는 것이다. Stall을 최소로 하고, 기능에 문제가 없다면, 이는 성능이 높다고 할 수 있다. 두번째는 분기 예측의 실패 가능성을 아예 없애는 것이다. 분기 예측의 실패를 줄이기 위해서는 높은 정확도를 가진 Branch prediction을 사용하면 된다. 하지만, 여전히 실패 가능성은 존재한다. 이를 해결하기 위해서, Branch 를 Decode 단계에서 끝낼 수 있도록 설계 하면 된다. Branch 를 Decode 단계에서 끝낼 수 있도록 설계하면 Decode 단계에서 Branch 분기 여부를 결정하고 결정된 PC 값에 따라 Fetch를 진행 하도록 하면, Branch predict를 할 필요가 없게 된다. 이를 그림으로 표현 하면 다음과 같다.



MicroArch 관점에서 Decode를 실행할 때, Fetch는 Decode가 끝날 때까지 기다린 후 실행되게 해야 한다. Branch 가 Decode 되어 Branch 여부가 결정이 나고, Branch 여부에 따라 다음 Fetch 될 PC값을 정할 수 있다. 또한, Branch와 함께 JR JAL J 명령어들 또한 Decode 단계에서 주소 값 처리를 완료한 후 PC 값을 갱신한다. 위 방법을 통해

하지만, 이런 방식으로 Branch와 Jump 명령어들을 Decode 단계 에 옮기게 되면 Data Hazard 가 발생하게 된다. 이에 대한 처리는 아래 Hazard 처리에 서술 되어 있다.

2) 코드 전체 구조

Main 함수에서 하는 작업들은 Single Cycle 과 동일하며, main 함수에서 binary file을 읽고 레지스터와 메모리를 초기화 한다. DataPath 함수에서 Latch와 Control unit 을 malloc을 통해 만들어준다. 이후 while 루프에 들어가면 Hazard detection 을 수행하고 수행된 결과를 토대로 write back 부터 fetch까지 역순으로 진행된다. Dataparh 함수 안에 있는 ctrl_flow라는 배열이 각 단계가 시행되는지 여부를 담고있다. 만약, ctrl_flow 값이 0이면 nop이며 1이면 단계를 실행한다. ctrl_flow가 -1인 경우, 파이프라인의 처음과 끝에 있는 아무것도 실행되지 않는 단계이다.

전체적인 코드의 구조는 다음과 같다.

main	1. Read & Save binary file, Initialize register & memory
data path	initialize the latch & control unit, ctrl_flow, move to loop
Loop	3. Hazard Detection : EX&MA, lui Hazard, lw, branchJr Hazard
Loop	4. Loop : writeback -> memory access -> execute -> decode -> fetch
data path	5. Print significant instruction count
main	6. back to main

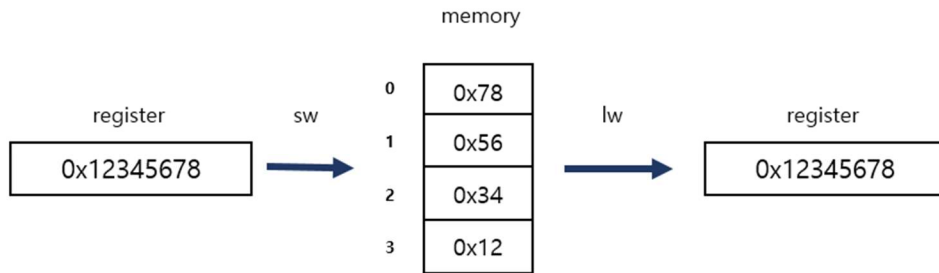
3) Pipeline 구현의 세부적인 요소

1. Memory 와 Register의 크기 구현

실제 현실에서 Memory는 1byte로 구성된 하나의 긴 메모리다. 이를 구현하기 위해서 uint8_t(1byte unsigned int) 타입으로 구현하였다. 또한 레지스터는 uint32_t(4 bytes unsigned int) 데이터 타입으로 구현 하였다. Unsigned로 구성한 이유는 C언어에서 int로 저장하면 최상위 비트에 따라 값이 변하게 된다. 메모리에 값을 저장할때, uint8_t 값에 uint32_t값을 나누어 저장하는데, unsigned int가 아니라면 값이 변화 되기 때문에 Unsigned를 사용하였다.

2. 리틀 엔디안

레지스터 값을 메모리로 저장 할 때, 레지스터의 크기와 메모리 크기가 같지 않다. 따라서, 분할하여 저장되어야 한다. 메모리에 저장하는 방식에는 2가지로 빅엔디안과 리틀엔디안이 있다. 리틀엔디안의 경우, 메모리의 주소중 하위 값에 register의 하위 비트값을 저장 하는 방식이며 이를 그림으로 표현하면 다음과 같다.



구현된 리틀엔디안의 코드는 다음과 같다.

[Store]

```

else if (ex_mem->ctrl->mem_write == 1) { // sw
    //printf("sw : store val 0x%x -> Mem[0x%x]", ex_mem->mem_val, ex_mem->alu_res);
    ma_count++;
    for (int i = 0; i < 4; i++) {
        Memory[ex_mem->alu_res + i] = (ex_mem->mem_val >> 8 * i) & 0xff;
    }
}

```

[Load]

```

ma_count++;
uint32_t temp = 0;
for (int j = 3; j >= 0; j--) {
    temp <<= 8;
    temp |= Memory[ex_mem->alu_res + j];
}
mem_wb->mem_read = temp; // lw로
//printf("0x%x val -> reg[0x%x]", mem_wb->mem_read, ex_mem->reg_idx);

```

3. 비트 연산

명령어는 4byte로 이루어져 있으며, 명령어로부터 레지스터 번호와 immediate, funct 값을 알 수 있다. shift 연산자와 비트 연산자 &를 통해 값을 추출 할 수 있으며 다음 그림을 통해 각각의 추출 방법을 알 수 있다.

	shift right	and(&) with	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
address	0	0x3ff.ffff					1				1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	
rs	21	0x1f					1				1	1	1																						
rt	16	0x1f									1				1	1	1	1																	
rd	11	0x1f																	1	1	1	1	1	1											
imm	0	0xffff																	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	
funct	0	0x3f																									1				1	1	1	1	1
shamt	6	0x1f																					1				1	1	1						
opcode	26	none	1	1	1	1	1				1																								

4. Hazard 처리 방법

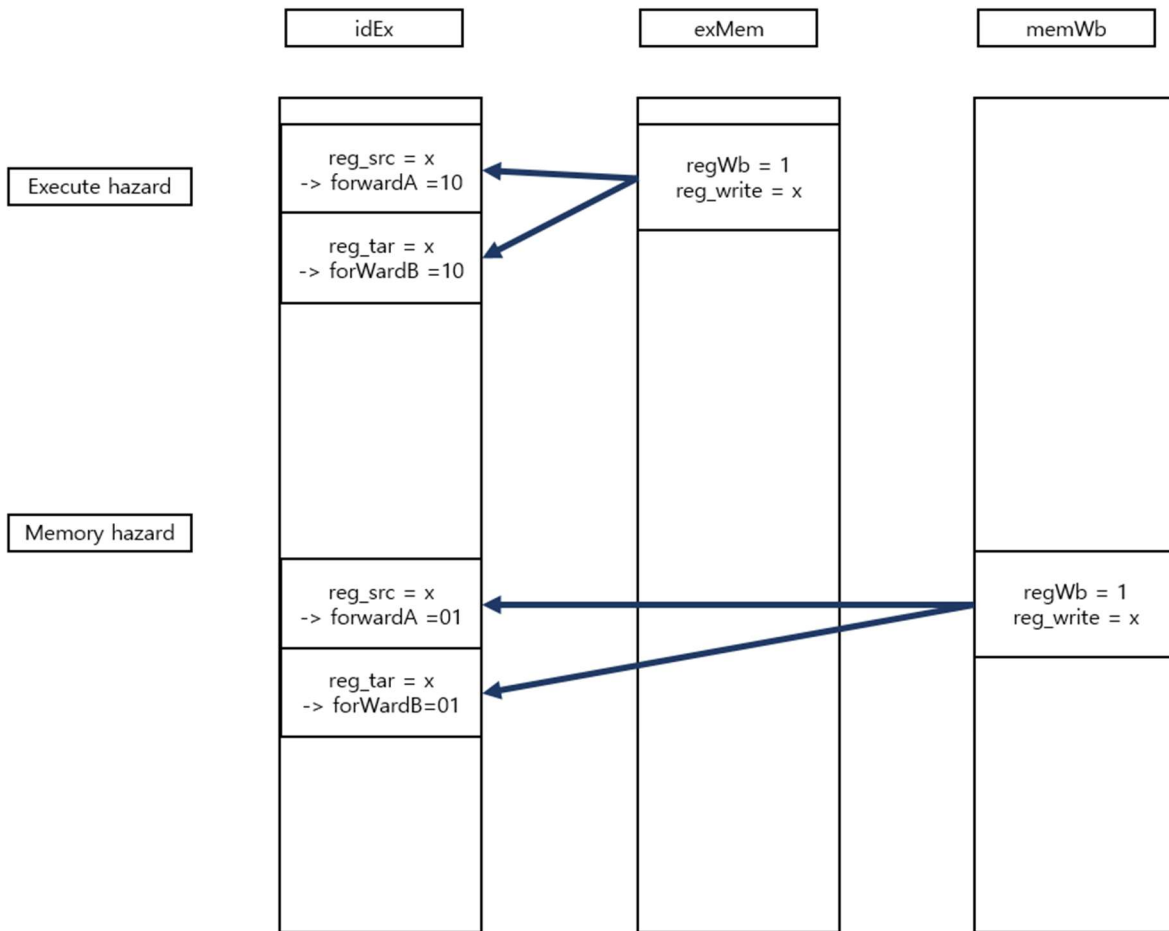
구현한 Hazard 처리는 Forwarding으로만 구성되어 있다. Decode 단계로 Branch와 Jump 관련 명령어들을 옮기면서 Control Hazard에 대한 처리는 불 필요 해졌지만, 이로 인해 Data Hazard가 생겼다. Jump 와 Jr은 jumpaddr 를 명령어를 통해 받기 때문에 Data Hazard는 없다. 하지만, Branch와 JR 명령어의 경우, 각각 rs와 rt값이 변하면 안되기 때문에 forwarding이 요구된다. 이 두 명령어 관련된 Hazard를 BranchJr hazard 이라고 하겠다.

Hazard 총 수는 총 4종류이다.

1. execute hazard
2. memory access hazard
3. branchJr execute hazard
4. branchJr memory access hazard

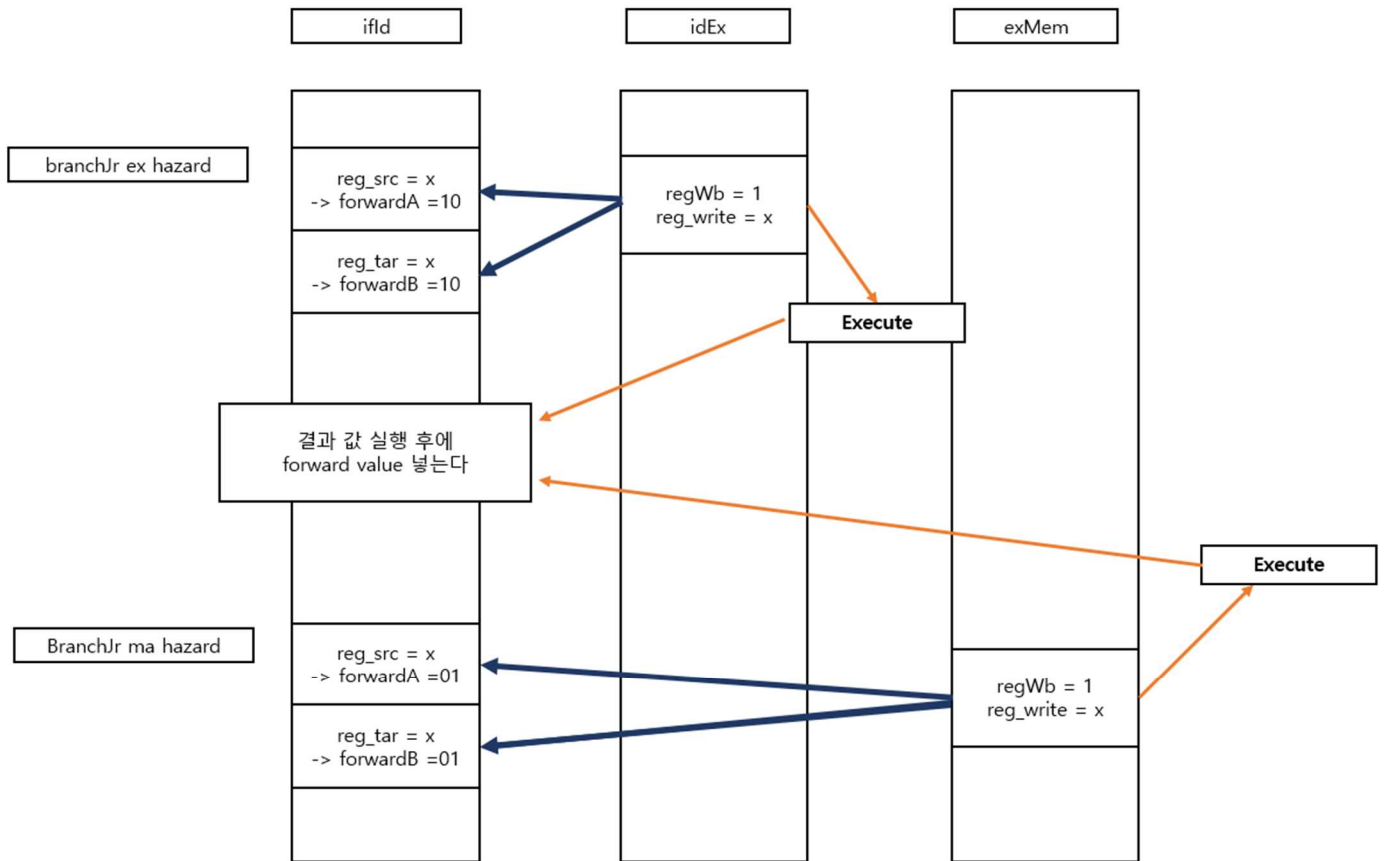
위 hazard 처리를 분류하면 1번과 2번은 idEx latch에서 reg_src나 reg_tar의 값이 memWb , exMem latch의 reg_write index와 같다면 실행된다.

3번과 4번은 branch나 Jr이 decode 전 ifId latch의 reg_src나 reg_tar 값이 idEx와 exMem 의 reg_write index와 같다면 실행된다. 2번과 4번은 각 latch에 해당되는 단계를 진행한후 alu result나 mem_read값을 받는 형태로 진행된다. 자세히 말하면, idEx 의 reg_write과 ifId의 reg_src,tar 값과 같을 때, idEx의 명령어를 execute 시키고 나온 alu_result 값을 다시 idEx의 reg_write에 넣어주는 방식이다. 1번부터 4번까지 그림으로 표현 하면 다음과 같다.



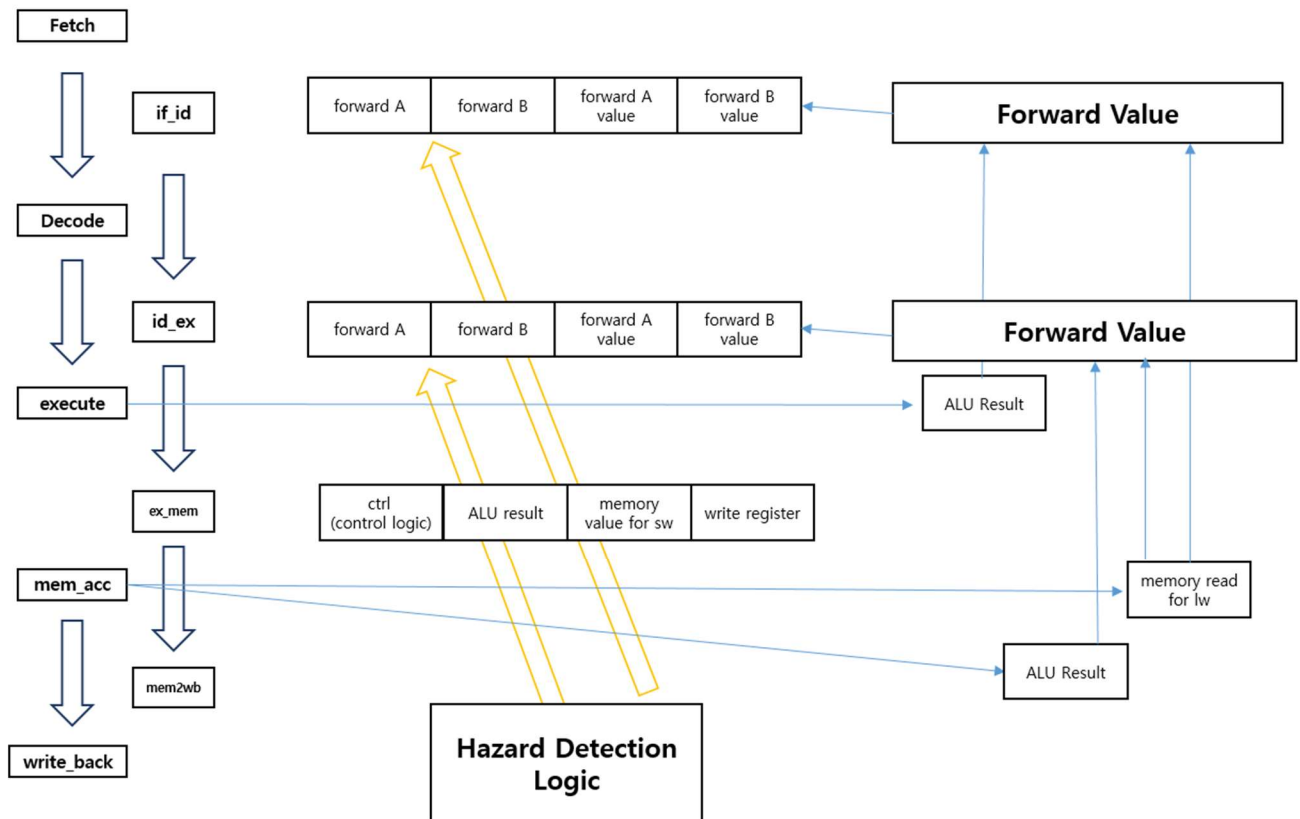
위 그림에서 x는 0이 아닌 경우이다.

1번과 2번의 경우이며 forward 값이 10 이면 exMem의 alu result 값을 받으며 01인 경우 memWb의 값을 받는다. 또한 forward_val 라는 변수를 latch안에 만들어 놓았으며, forward 값에 따라 들어가는 값이 다르게 설정하였다, forWardA 값이 1보다 크다면, regSrc가 최신 값을 받아야한다. forwardB 값이 1보다 크면, regTar 이 최신 값을 받으면 된다. forward값이 01인 경우 exMem이 근원지 이며 forward 값이 10인 경우 memWb이 근원지이다. 만약, LW가 1번 Hazard에 있는 경우, exMem latch에서 mem_read 값이 없으므로 즉시 줄 수 없는 문제가 생긴다. 이 경우, LW를 **선행적으로** 실행시켜 mem_read 값을 갱신된 memWb Latch 에서 얻어주면 된다. 제시하는 **선행적 실행**은 Branch JR Hazard 에서 더 큰 힘을 발휘한다.



위 그림은 branchJr hazard 를 해결하는 forwarding이다. 1번과 2번과 유사하며, 단계를 앞으로 한단계 당겼다. 위 Hazard에서 exMem latch에서는 Alu_result 값을 받을 수 있지만, idEx latch에서는 Alu_result를 받을 수 없는 문제가 생긴다. 따라서, 3번의 경우 execute를 **선행적으로** 실행한 후 최신화 된 exMem latch에서 Alu_res 값을 받도록 한다. 만약 ifId에 Branch나 JR 이 사용되며, exMem에서 LW가 존재하는 경우, memory access 단계를 **선행적으로** 실행한 후 mem_read 값을 받아서 ifID latch의 forward_value 값을 갱신 하면 된다.

아래 그림은 위 설명을 정리한 도표이다.



IV. 코드 결과 및 결과에 대한 수학적 고찰

1) 코드 결과 값

[simple]

```
=====
Return register (r2)      : 0
Total cycle               : 8
r-type count             : 5
i-type count             : 4
j-type count             : 0
branch count             : 0
memory access count      : 2
nop count                : 2
register write count      : 5
=====
```

[simple2]

```
=====
Return register (r2)      : 100
Total cycle               : 10
r-type count              : 5
i-type count              : 7
j-type count              : 0
branch count              : 0
memory access count      : 4
nop count                  : 1
register write count      : 7
=====
```

[simple3]

```
=====
Return register (r2)      : 5050
Total cycle               : 1330
r-type count              : 106
i-type count              : 920
j-type count              : 1
branch count              : 102
memory access count      : 613
nop count                  : 306
register write count      : 715
=====
```

[simple4]

```
=====
Return register (r2)      : 55
Total cycle               : 243
r-type count              : 62
i-type count              : 153
j-type count              : 11
branch count              : 10
memory access count      : 100
nop count                  : 20
register write count      : 151
=====
```

[fib]

```
=====
Return register (r2)      : 55
Total cycle               : 2679
r-type count             : 548
i-type count             : 1697
j-type count             : 164
branch count             : 109
memory access count      : 1095
nop count                 : 273
register write count      : 1530
=====
```

[gcd]

```
=====
Return register (r2)      : 1
Total cycle               : 1061
r-type count             : 224
i-type count             : 637
j-type count             : 65
branch count             : 73
memory access count      : 486
nop count                 : 138
register write count      : 595
=====
```

[fibonacci.bin] : New binary file : 16번째 피보나치 수열 값

```
=====
Return register (r2)      : 610
Total cycle               : 46372
r-type count             : 9868
i-type count             : 29601
j-type count             : 1973
branch count             : 2960
memory access count      : 18747
nop count                 : 4933
register write count      : 26639
=====
```

[input4] 성공 : Pipeline

```
=====
Return register (r2)           : 85
Total clock cycle              : 23372706
r-type count                   : 5076370
i-type count                   : 11190042
branch, j-type count, jr      : 2029805
lw count                      : 6090476
sw count                      : 1026130
nop count                     : 5076495
register write count           : 15238464
=====
```

2) 코드의 결과 값에 대한 성능 계산과 고찰

[input4를 기준으로 계산]

single Cycle의 결과는 다음과 같다.

```
=====
Return register (r2)           : 85
Total cycle                    : 23372706
r-type count                   : 5076368
i-type count                   : 13219741
j-type count                   : 103
branch count                   : 2029699
memory access count           : 7116606
nop count                     : 5076494
=====
```

코드의 목적은 최대 성능을 가진 Pipeline을 구현하는 것이다. 따라서, 이번 코드는 단 하나의 stalling을 구현하지 않았으며 오직 forwarding을 통해서만 pipeline hazard를 처리했다. 또한, branch predict를 실패할 수 있는 부분도 Branch를 decode 단계로 옮김으로써 실패할 가능성을 아예 지웠다. 그 결과, Single Cycle과 Pipeline 코드의 결과가 동일했다. 물론, C코드는 sequential한 flow를 가진다. C코드는 병렬적인 처리를 할 수 없기에 Pipeline 코드는 Single Cycle 코드의 배열만 바뀐 형태로도 해석할 수 있다. 하지만, 이번 과제의 목적은 현실적 구현이 아닌 개념적 구현이기 때문에, 이에 대한 문제는 건너 뛸 수 있다.

만약 위 결과를 토대로 파이프라인과 Single Cycle의 성능 계산을 한다면 다음과 같다.

=====

Fetch : 200ps

Decode : 100ps (branchJr은 300ps)

EX : 200ps

MA : 100ps

WB : 200ps

이를 정리하면

memory access : 800ps

r-type : 600ps

branch를 제외한 i-type : 600ps

Branch Jr J-type : 500ps

=====

Single Cycle의 경우 하나의 Cycle이 돌아가는 시간은 가장 긴 LW를 기준으로 800 ps으로 돌아가므로

총 실행 시간 (싱글 사이클)= $23372706 \times 800\text{ps}$ 이 걸린다.

파이프라인의 경우

$$\begin{aligned}
\text{총 실행 시간 (싱글 사이클)} &= 5076370 \times 600\text{ps} + 11199042 \times 600\text{ps} + 2029805 \times \\
&500\text{ps} + 6090476 \times 800\text{ps} + 1026130 \times 800\text{ps} \\
&= 3045822000\text{ps} + 6719425200\text{ps} + 1014902500\text{ps} + 4872380800\text{ps} + 718291000\text{ps} \\
&= 16370821500\text{ps}
\end{aligned}$$

위 같은 시간이 걸리며 다섯 단계가 한번에 진행되므로 최소 3274164300ps (이론 상 모든 파이프라인이 stall 없이 진행 되었 을때) 의 실행시간을 가진다. 이론 상 가능한 시간으로 성능 비교를 하면 다음과 같다.

$$23372706 \times 800\text{ps} / 16370821500\text{ps} = 1.14$$

$$23372706 \times 800\text{ps} / 3274164300\text{ps} = 5.7$$

이때, 파이프라인은 최소 1.14배 빠르며 이론상으로는 5.7배 Single Cycle 보다 빠르다.

결론

이번 과제를 통해 Pipeline과 Mips Architecture에 대한 이해를 심화 시킬 수 있었다. 파이프라인 기술은 현대 컴퓨터 구조에 성능을 향상 시키는 중요한 기술이다. 본 보고서를 통해 Pipeline의 설계와 싱글 사이클 프로세서의 성능을 비교 분석해 보았으며 각각의 특징과 한계를 알아보았다. 이번 과제에서 가장 힘들었던 부분은 디버깅이었다. Pipeline 로그가 10GB가 넘어가서 로그를 다 뒤져 볼 수 없어 논리를 다시 바꾸는데 더 큰 시간을 들였다. 이를 통해, 앞으로 코딩을 하면서 디버깅에 대한 방향과 생각을 건고히 하는 계기가 되었다.

[그림 출처]

Copyright 2009 by Elsevier, Inc., All rights reserved. From Patterson and Hennessy, Computer Organization and Design, 4th ed.

[빌드]

VS 2022

로그가 아닌 결과값을 보기 위해서는 `ctrl + f` 를 하여 `printf(` 를 `//printf(` 로 모두 바꾸기를 한 후 볼 수 있다.